

Stream Method	What You CAN Do	What You SHOULD NOT Do	Code Example (Valid)	Why / Notes
filter()	✓ boolean checks ✓ null checks ✓ comparisons ✓ method calls returning boolean	✗ modify external variables ✗ I/O operations ✗ object mutation	<code>.filter(e -> e.getAge() > 18 && e.isActive())</code>	Must be pure predicate
distinct()	✓ rely on equals() / hashCode() ✓ immutable objects	✗ mutable fields affecting equality	<pre>class User { public boolean equals(Object o) { ... } } stream.distinct();</pre>	Internally uses HashSet
map()	✓ transform objects ✓ create new objects ✓ call pure functions	✗ mutate input object ✗ change shared state	<code>.map(u -> u.getName().toUpperCase())</code>	One-to-one transformation
flatMap()	✓ return Stream ✓ split data ✓ expand collections	✗ return null ✗ heavy logic inside mapper	<code>.flatMap(s -> Arrays.stream(s.split("\\ ")))</code>	One-to-many mapping
sorted()	✓ natural ordering ✓ custom comparator ✓ multi-field compare	✗ comparator with side effects	<code>.sorted(Comparator.comparing(User::getAge))</code>	Stateful + expensive
peek()	✓ logging ✓ debugging ✓ metrics	✗ business logic ✗ mutation	<code>.peek(e -> System.out.println(e))</code>	Debug only
collect()	✓ accumulate data	✗ non-thread-safe	<code>.collect(Collectors.toList())</code>	Terminal operation

	✓ grouping ✓ reduction ✓ custom collectors	mutation (parallel)		
--	--	---------------------	--	--

List<Employee> result =

```

employees.stream()           // Source
.filter(e -> e.salary > 50000) // FilterStage
.map(e -> new Employee(e.id, e.name, e.salary)) // MapStage
.distinct()                  // DistinctStage
.sorted(Comparator.comparingInt(e -> e.salary)) // SortedStage
.peek(e -> System.out.println("After sort: " + e)) // PeekStage
.collect(Collectors.toList()); // CollectOperation

```

System.out.println("Final Result: " + result);